

Lab 06: Functions in MIPS Assembly Programming

Assigned: Monday, May 4, 2026

Due: Monday, May 11, 2026

Points: 100

- You may collaborate on this homework with AT MOST one person, an optional "homework buddy".
 - MAY ONLY BE TURNED ON **GRADESCOPE**.
 - There is NO MAKEUP for missed assignments.
 - We are strict about enforcing the LATE POLICY for all assignments (see syllabus).
-

Goals for This Week

By the time you have completed this work, you should be able to call a defined function properly and be able to demonstrate how to work within the MIPS Calling Convention.

Provided skeleton files (on CSIL and on Canvas):

- `bubble.asm`
- `calculation.asm`
- `swap_case.asm`

Step by Step Instructions

You will perform 3 tasks. You may complete them in any order, though it is recommended to do them in the order presented (easiest to hardest). You may use any approved MIPS instructions or psuedoinstructions you want in order to implement these tasks. The initial step below describes how to get the files you will need into the appropriate place.

Remember that when writing assembly code for functions to always put the arguments into `$a0/$a1` registers and the return into the `$v0`. Look at the instructions below for further hints.

Initial Step: Create a Directory for This Lab and Copy Over the Files

You may either get the skeleton files from the Canvas assignment or you may copy them over from my directory on CSIL to yours.

If you chose to do the latter, then after you log into CSIL, go into your `cs64` directory that you created last time. Then create a new directory for this lab: `hwk6` and then go into that directory. Finally, copy over all of the files necessary for this week's tasks (there are 3 skeleton files):

```
cp ~zmatni/public_html/cs64s26/hwk6/* .
chmod +x ./*
```

Note the use of the trailing `.` in the above command, which stipulates that the specified files should be copied into the current directory. Also note that you may have to change file permissions once you copy them over using the `chmod` command.

Again, you have to modify all 3 files and keep their names the same for when you submit them on Gradescope at the end.

Task 1: Bubble Sort and Print (worth 30 pts)

Involves skeleton file (MIPS assembly): *bubble.asm*

You will write a MIPS assembly program that does a bubble-sorting on an array of integers and then prints it. **Your task then is to write the functions for bubble sort and print array in MIPS assembly and integrate and test them in the skeleton file we have provided.**

Recall that bubble-sort and print an array work as per the following C++ code (this is how we want you to implement them in assembly):

```
void bubble(int *array, int size) {
    int temp;
    for (int i = size-1; i >= 0; i--) {
        for (int j = 1; j <= i; j++) {
            if (array[j-1] > array[j]) {
                temp = array[j-1];
                array[j-1] = array[j];
                array[j] = temp;
            } // if
        } // for j
    } // for i
    return;
}

void printArray(int *array, int size) {
    for (int i = 0; i < size; i++){
        cout << array[i] << " ";
    }
    cout << endl;
    return;
}
```

Note that these functions work for any integer array with an array size > 1.

The program in assembly could be set up in the beginning like this example:

```
.data
array: .word 6, 4, 0, 1, 2, 9, 3, 5, 8, 7
alen: .word 10
newline: .asciiz "\n"
space: .asciiz " "
```

But remember that the program should work for a different integer array of any other length > 1.

Note that newline and space are defined since they're used in the **printArray()** function in the C++ code above (and yes, we'd like you to do it the same way).

The rest of the assembly program could look like this:

```
.text
bubble:
    # CODE MISSING: Student to complete this part

printArray:
    # CODE MISSING: Student to complete this part

main:
# Prints an integer array of length alen before and after bubble sort
    la $a0, array
    la $a1, alen
    jal printArray

    la $a0, array
    la $a1, alen
    jal bubble

    la $a0, array
    la $a1, alen
    jal printArray

    li $v0, 10
    syscall
```

So, with the array example given above, the program should print:

```
6 4 0 1 2 9 3 5 8 7
0 1 2 3 4 5 6 7 8 9
```

Note there IS a newline character at the end of each printed line, as well as space characters between every printed element (including at the end). Again, this adheres to the C++ code for **printArray()** shared with you above.

For this task, you only need to adhere to the simple MIPS Calling Convention (i.e., you are not expected to access the stack at all, you don't have to worry about preserved/unpreserved register values). Simply make sure you pass arguments in **\$a0** and **\$a1**. Since both functions here are **void** types, you are not returning any values. Make sure you use **jal** and **jr \$ra** properly!!

A skeleton assembly program that you will complete to write your code is available to you in the **bubble.asm** file.

IMPORTANT: While we provide an array in the file, your code should work on **any arbitrary array of any arbitrary length (strictly larger than zero)**. On our end, for testing purposes, we may use different int arrays, so your code should work with any other valid array (size > 1).

Task 2: Calculation Program (worth 30 pts)

Involves skeleton file (MIPS assembly): *calculation.asm*

Write a MIPS assembly program that mimics this C++ code which includes TWO function calls (your solution MIPS assembly program MUST also include ALL of the functions/function calls shown here):

```
int remove(int a, int b) {
    b -= a;
    return b;
}
int calc(int x, int y, int n) {
    int z = 5;
    for (int i = 0; i < n; i++) {
        z = z - x + 2*y;
        if (x >= 2)
            y = remove(x, y);
        x += 1;
    }
    return z;
}

// You don't need to implement main, our "main" will call "calc()" for you.
// It is just included here so you can see how the function is called.

int main() {
    int a = 4, b = 10, n = 3;
    cout << calc(a, b, n);
}
```

For this task, you need to adhere to the MIPS Calling Convention (i.e., figure out if you have to access the stack and think about preserved/unpreserved register values). Of course, always make sure you pass arguments into functions in the **\$a**-registers and return values in **\$v0**. Make sure you use **jal** and **jr \$ra** properly!!

A skeleton assembly program that you will complete to write your code is available to you in the **calculation.asm** file.

An example run with inputs 4, 10, 3 for a, b, n (respectively) should look like this:

24\$

Note there is no newline character at the end (the \$ symbol indicates the user prompt in UNIX).

Task 3: Swap Case (worth 40 pts)

Involves skeleton file (MIPS assembly): `swap_case.asm`

In this task, you will implement the **SwapCase** function in `swap_case.asm` using the **MIPS Calling Convention** as we learned it in lecture. The function **SwapCase** takes one argument: The address of a string of characters, held in `$a0`.

The main purpose of the function is to swap the *case* of each character in the string. This is achieved by going through the string one character at a time. On each character, the function should do the following, in order:

1. Check if the character is a letter or not, if yes then continue doing next steps or else skip to the next character i.e., go to the next loop iteration in step 5.
2. Print out the *current* character of the string with a newline.
3. Print out the *new* character with the *opposite* case. (e.g., if character was **a**, then **A** should be printed and vice versa), with a newline.
4. Call the already-existing **ConventionCheck** function, to help make sure the proper MIPS Calling Convention is being followed.
5. Iterate the loop.

Do not implement the “ConventionCheck” function yourself, or modify it in any way. On our end, we are going to use our own version of **ConventionCheck** while grading, so the only assumptions you can make about **ConventionCheck** are that it will follow the **MIPS Calling Convention**.

In parts of the code where you are *examining* each *character* from the string, you may have to use the core instructions **lb** and **sb**, instead of **lw** and **sw**. These work the same way as **lw** and **sw**, except that, instead of operating on *words* (32-bit values, like integers), they operate *bytes* (8-bit values, like characters).

As mention, you **will** have to make sure that you are adhering to the FULL MIPS convention check, so use of the stack is a given. Hint: When using the stack, you should still use **lw** and **sw** for popping and pushing.

While we provide a string in the `swap_case.asm` file, your code should work on **any** arbitrary non-zero-length string. On our end, for testing purposes, we will use different strings, so your code should work with any other valid string.

The main function provided in `swap_case.asm` calls **SwapCase** on your behalf, and it will display the contents of the string after calling **SwapCase**.

Your output should have the format shown below. The first two lines and the last line are generated by the **main** function, so you do not have to implement anything to print those portions out. The lines containing the words “**Convention Check**” are produced by the **ConventionCheck** function that you must call after printing each input.

To help clarify which output you need to produce and which output is produced for you, the output that you must produce is highlighted.

```
Enter string:
Hi! This is cool.
H
h
Convention Check
i
I
Convention Check
T
t
Convention Check
h
H
Convention Check
f
F
Convention Check
s
S
Convention Check
f
F
Convention Check
s
S
Convention Check
c
C
Convention Check
o
O
Convention Check
o
O
Convention Check
l
L
Convention Check
Output:
hI! tHIS IS COOL.
$
```

Note there IS a newline character at the end (the \$ symbol indicates the user prompt in UNIX).

Remember to follow the MIPS Calling Convention while implementing your SwapCase function. This means remembering what to do with preserved and unpreserved registers. Otherwise, the program may crash hard, and you will receive no credit.

Turn in Everything Using Gradescope

Test ALL your programs extensively (we certainly will) before you turn them in!

You are going to turn in **THREE** things into Gradescope, that is, the completed 3 assembly program files: **calculation.asm**, **bubble.asm**, and **swap_case.asm**. **MAKE SURE THAT YOUR FILENAMES MATCH THESE** otherwise the autograder will not work (and your score will show a zero).

Navigate to the Lab assignment **assignment06** on Gradescope and upload the files. *Every student must upload their own code.* We will check for plagiarism and permitted use (i.e. as taught in this class) of MIPS assembly code.